

מבוא לתכנון וניתוח אלגוריתמים

תכנון וניתוח אלגוריתמים - תרגול ראשון
רון פינקלשטיין, סמסטר תשפ"ו א' (אתגר)
המצגות באדיבות אלון קרימגנד ואורן דובין

בירוקרטיה

- **מרצה** דר' עמית לוי alevi@cs.Haifa.ac.il
- **מתרגל** רונן פינקלשטיין finkelstein.4ronen@gmail.com
- **שעות קבלה** בתיאום מראש, בזום
- **תרגילי בית** אחת לשבועיים. הגשה בזוגות, בונוס הקלדה 5 נקודות.
- **ציון סופי** – 85% מבחן, 15% תרגילי בית (מגן)

הקדמה

מהו אלגוריתם?

- אלגוריתם היא דרך שיטתית שבה בהינתן קלט כלשהו, מבצעת עליו פעולות ומחזירה עליו פלט
- בקורס נתעסק באלגוריתמים **דטרמיניסטיים** בלבד: על קלט מסוים תמיד יפלטו את אותו הפלט



אלגוריתם

- בקורס, כאשר נדבר על אלגוריתם כלשהוא, בעצם נדבר על שלושה דברים עיקריים:
 - **תיאור האלגוריתם**, שלב אחר שלב
 - טענה על **סיבוכיותו** של האלגוריתם
 - טענה על **נכונותו** של האלגוריתם
- האלגוריתם שנדבר עליהם בקורס מחולקים לשתי קטגוריות עיקריות:
 - **אלגוריתמי הכרעה**
 - האם יש דרך לעשות...? אם כן, מהי הדרך...?
 - האם גרף קשיר? האם ישנו מסלול בין שני צמתים? מהו המסלול?
 - **אלגוריתמי אופטימיזציה**
 - מהי הדרך הטובה ביותר לעשות...?
 - מהו מספר הצעדים המינימלי? מסלול קצר ביותר בין שני צמתים?

תיאור אלגוריתם

האם y מתחלק ב- x ?

- If $x = 0$:
 - Return false.
 - Else:
 - $y \leftarrow y \% x$
 - If $y = 0$:
 - Return True.
 - Else:
 - Return False.
- דרך נפוצה לתיאור אלגוריתמים היא בעזרת **פסודו-קוד** (Pseudo-Code). כתיבת פסודו-קוד אמורה להיות נוחה למתכנתים, שכן היא מזכירה קוד בשפות תכנות, אך ללא חוקי דקדוק ברורים וחד משמעיים. מספיק שה"קוד" יהיה מובן לאדם שקורא אותו כדי שיהיה תקין.
 - עם זאת, לעיתים אלגוריתם הכתוב בפסודו-קוד יכול להיראות מסורבל וקשה להבנה. לכן, **ניתן לתאר אלגוריתמים גם במילים, כמו סיפור**. תוכלו להסביר שלב אחר שלב מה אתם עושים עם הקלט ואיך אתם מגיעים לפלט.

הוכחת נכונות אלגוריתם

- ישנן כמה שיטות נפוצות להוכחת נכונות האלגוריתם שלכם.
- באופן כללי, תרצו לשכנע את הקורא כי אין באלגוריתם "חורים", וכי בעבור כל קלט תקין יפלוט את הפלט הרצוי.
- **הוכחה ישירה - על קלט אלמוני**
 - ניקח קלט אלמוני כלשהו ונכניס לאלגוריתם.
 - בעזרת כלים מתמטיים, נחשב צורה כללית לפלט הרצוי של האלגוריתם.
 - מראים כי הפלט של האלגוריתם זהה לאחד שחישבנו והוכחנו כנכון.
- **הוכחה בבנייה - אינדוקציה על צעדי האלגוריתם**
 - תחילה מוכיחים כי המצב ההתחלתי של האלגוריתם תקין.
 - מוכיחים כי כל צעד שהאלגוריתם עושה, או כל פיסת קלט חדשה, משמרת את מצב האלגוריתם תקין.
 - מוכיחים לבסוף כי האלגוריתם עוצר בשלב הנכון, ומחזיר את הפלט הרצוי בהתאם למצב.

טיפים לפתרון בעיות

- **שחקו עם הקלט שלכם.** מה האלגוריתם אמור להחזיר בעבור מקרים מיוחדים? נסו למצוא תכונות שאתם רוצים שהאלגוריתם יקיים.
- בעבור אלגוריתם המקבל מספר, נסו להבין בעבור אילו מספרים הבעיה קלה יותר. ראשוניים? זוגיים?
- בעבור מערך, מה עם האיברים כולם שווים? מערך מונוטוני? חיוביים/שליליים בלבד?
- בעבור עצים, מה האלגוריתם אמור להחזיר בעבור כוכב או חוט? עץ בינארי, עץ בינארי מושלם, וכו'.
- בעבור גרפים כלליים, מה קורה כאשר הגרף הוא עץ? כאשר הוא DAG? קליקה?
- התחילו מהפתרון הטריטוריאלי, ונסו **למצוא בו חולשה.**
- הרבה אלגוריתמי הפרד ומשול ותכנות דינאמי משתמשים בפתרון הטריטוריאלי עם שיפור קל שמשפר משמעותית את הסיבוכיות. בהפרד ומשול, אנו מחלקים את הבעיה לתתי בעיות שיותר קל לפתור בנפרד, ובתכנות דינאמי אנו מנעים מלחשב תתי בעיות יותר מפעם אחת.
- נסו לבנות **הבחנות** שיקדמו אתכם לפתרון.
- הבינו את הבעיה. ממה מורכב הפתרון? ישנה חוקיות מסוימת שהפתרון תמיד מכיל?

טיפים לפתרון בעיות

- נסו **לנחש** את סיבוכיות הפתרון הגיונית, ובהתאם לחשוב על אלגוריתם.
- פתרונות בסיבוכיות $O(n)$ מכילים בדר"כ מעבר על כל הקלט וביצוע $O(1)$ פעולות בעבור כל איבר.
- פתרונות בסיבוכיות $O(\log n)$ מרמזים על חיפוש בינארי, או הפרד ומשול: חילוק הבעיה לתתי בעיות.
- פתרונות בסיבוכיות $O(n \log n)$ בדר"כ מכילים מיון כלשהו, או חיפוש בינארי על כל אחד מאיברי הקלט.
- פתרונות בסיבוכיות $O(\sqrt{n})$ מרמזים על הפרד ומשול: חלוקת הקלט ל- $\Theta(\sqrt{n})$ קבוצות, כאשר גודל כל קבוצה $\Theta(\sqrt{n})$.
- אם אתם לגמרי תקועים, עברו על "**ארגז הכלים**" שברשותכם:
- שיטות ודרכים לפתרון בעיות שנלמד בקורס, כמו פתרונות חמדניים, הפרד ומשול, תכנות דינאמי. אולי הפתרון משתמש בשיטה אחת או יותר.
- אלגוריתמים ידועים שונים שנלמד בקורס, כמו מיון טיפולוגי, זרימה מקסימלית או DFS. יתכן שהפתרון משתמש באלגוריתמים אלו ישירות, או שתוכלו להתאים את האלגוריתם לבעיה בעזרת שינויים קלים.

סיבוכיות זמן ריצה

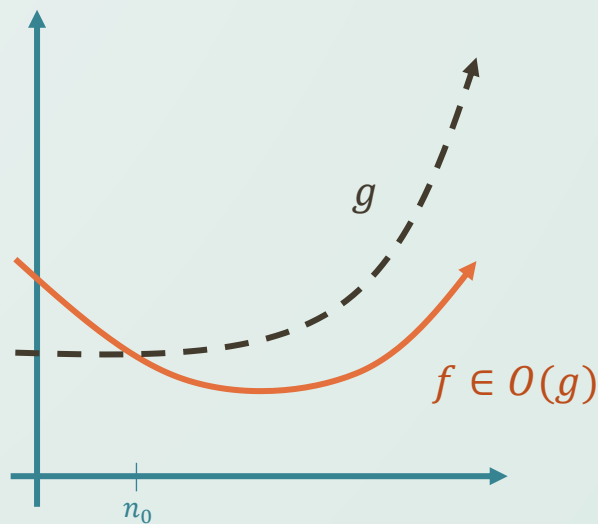
Running Time Complexity and Asymptotic Order of Growth

סיבוכיות

$$f \in O(g) \iff \exists c, n_0 > 0 : \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)$$

$$f \in \Omega(g) \iff \exists c, n_0 > 0 : \forall n \geq n_0 : 0 \leq c \cdot g(n) \leq f(n)$$

$$f \in \Theta(g) \iff f \in O(g), f \in \Omega(g)$$



סיבוכיות

- הגדרות שקולות, הנובעות ישירות מהגדרת מושג הגבול:

$$f \in \mathcal{O}(g) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$$

$$f \in \Omega(g) \iff 0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

$$f \in \Theta(g) \iff 0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$$

סיבוכיות

- בנוסף, קיימות שתי דרכים יותר "חזקות" לתיאור יחס אסימפטוטי:

$$f \in o(g) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

$$f \in \omega(g) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

השוואת זמני ריצה

- טבלה הנועדה להשוות זמני ריצה של אלגוריתמים בסיבוכיות שונה בהתאם לגדלי קלט שונים.
- תחת ההנחה כי מחשב ממוצע יכול לבצע כמיליון פעולות בשנייה בודדת.

$n!$	2^n	1.5^n	n^3	n^2	$n \log_2 n$	n	
4 שניות	> 1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	$n = 10$
$\sim 10^{25}$ שנים	18 דקות	> 1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	$n = 30$
הרבה	36 שנים	11 דקות	> 1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	$n = 50$
הרבה	$\sim 10^{17}$ שנים	$\sim 10^4$ שנים	1 שניות	> 1 שניות	> 1 שניות	> 1 שניות	$n = 100$
הרבה	הרבה	הרבה	18 דקות	1 שניות	> 1 שניות	> 1 שניות	$n = 1,000$
הרבה	הרבה	הרבה	12 ימים	2 דקות	> 1 שניות	> 1 שניות	$n = 10,000$
הרבה	הרבה	הרבה	32 שנים	3 שעות	2 שניות	> 1 שניות	$n = 100,000$
הרבה	הרבה	הרבה	$\sim 10^4$ שנים	12 ימים	20 שניות	1 שניות	$n = 1,000,000$

תרגיל ראשון

- ברשותכם מחשב המסוגל לבצע 10^{10} פעולות בשנייה.
- בנוסף ברשותכם שישה אלגוריתמים. הביטויים מטה מתארים את מספר הפעולות המדויק שכל אלגוריתם יבצע בהינתן קלט באורך n .
- בעבור כל אחד מהאלגוריתמים, חשבו את גודל הקלט המקסימלי שבעבורו יסיים האלגוריתם את ריצתו בלכל היותר שעה.

$100n^2$	(ג)	n^3	(ב)	n^2	(א)
----------	-----	-------	-----	-------	-----

$n \log_2 n$	(ו)	2^{2^n}	(ה)	2^n	(ד)
--------------	-----	-----------	-----	-------	-----

תרגיל שני

- לפניכן חמש פונקציות $f_1, f_2, \dots, f_5$.
- סדרו אותן ברשימה כך שכל פונקציה f ברשימה שנמצאת אחרי g ברשימה מקיימת $g = \mathcal{O}(f)$.

$$f_1(n) = 10^n$$

$$f_2(n) = n^{1/3}$$

$$f_3(n) = n^n$$

$$f_4(n) = \log_2 n$$

$$f_5(n) = 2^{\sqrt{\log_2 n}}$$

קירוב סטרלינג

- לעתים קשה להתעסק עם הפונקציה $n!$. מסתבר שיש קירוב אסימפטוטי.

$$n! \sim \left(\frac{n}{e}\right)^n \sqrt{2\pi n} \quad \bullet$$

- כלומר הגבול של שתי הפונקציות שווה לאחד.

$$\lim_{n \rightarrow \infty} \frac{\left(\frac{n}{e}\right)^n \sqrt{2\pi n}}{n!} = 1 \quad \bullet$$

- הקירוב הזה שימושי מאוד בחישוב גבולות, והשוואת גדילה אסימפטוטית של פונקציות.
- הוכחת הקירוב יוצאת ממסגרת הקורס, למתעניינים יש הרבה הוכחות באינטרנט.

תרגיל שלישי

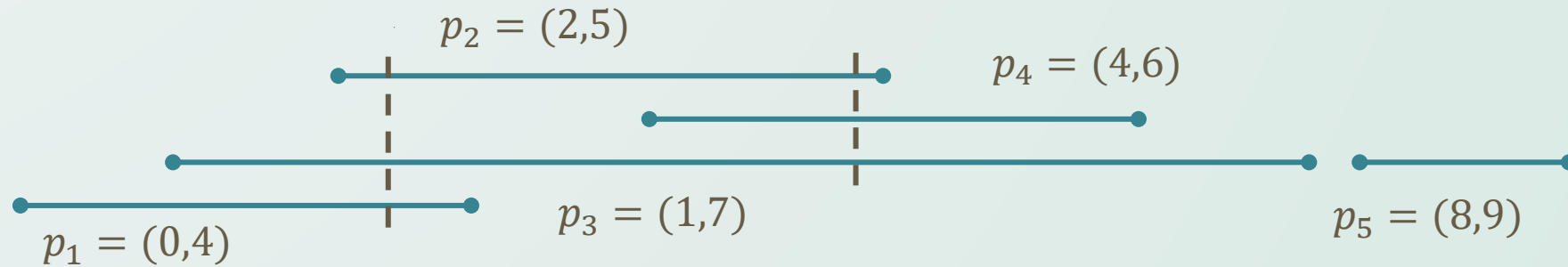
- סדרו את הפונקציות כמו קודם:

$$n^6, n^n, \binom{n}{n-6}, (6n)!, \binom{n}{n/6}$$

בעיית הסועדים

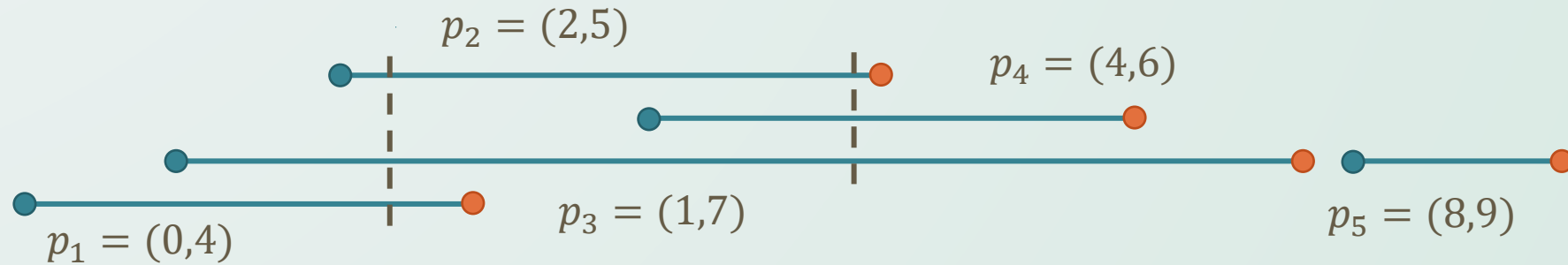
בעיית הסועדים

- אתם בעלים של מסעדה, וברשותכם רשימה של n זוגות **שונים** מהצורה $p_i = (a_i, b_i)$, המייצגים את זמני ההגעה והעזיבה של הסועדים במסעדה.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, למציאת **מספר הסועדים המקסימלי** בכל רגע נתון במסעדה. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



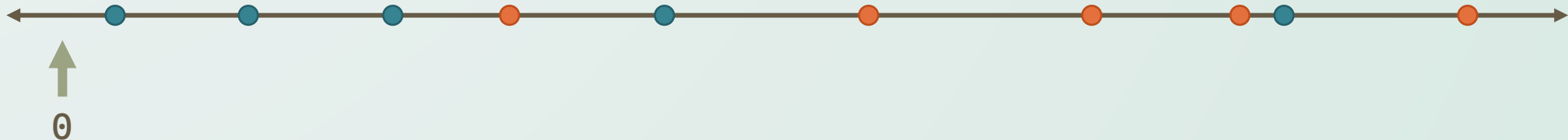
הבחנות בדרך לפתרון

- כל שינוי של מספר הסועדים במסעדה מתקיים באחת מ- $2n$ נקודות הקצה של האינטרוולים p_i .
- לכן בפרט, מספר הסועדים המקסימלי יבוא ישר כניסה של סועד, כלומר לאחר נקודה מהסוג a_i .
- זאת מכיוון שכל השינויים במספר הסועדים מתרחש לאחר נקודה כלשהי, ונקודות מהסוג b_i רק מורידות את מספר הסועדים, ולכן לא יתכן שאחריה מספר הסועדים יהיה מקסימלי.
- כל מה שנותר לעשות הוא לעקוב אחרי מספר הסועדים לאחר כל נקודה, ולקחת את המקסימום.



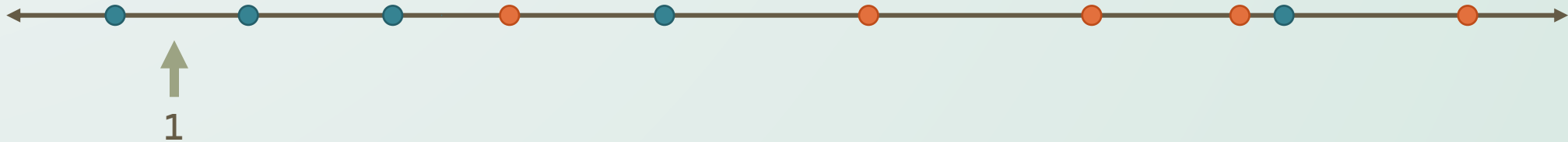
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



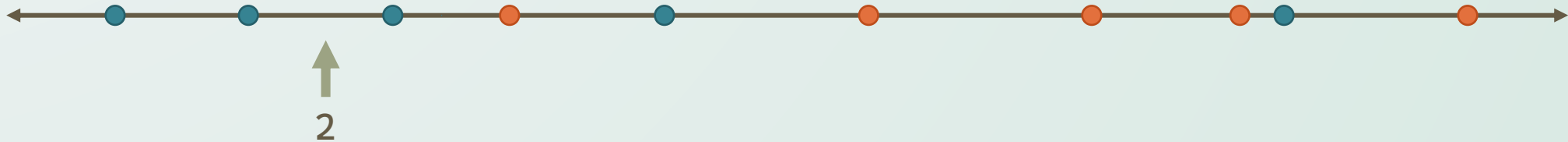
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



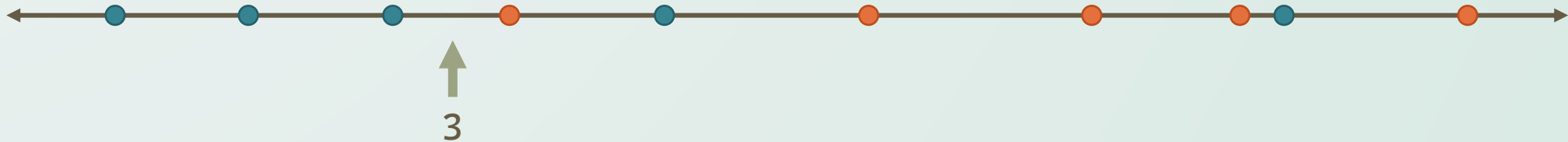
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



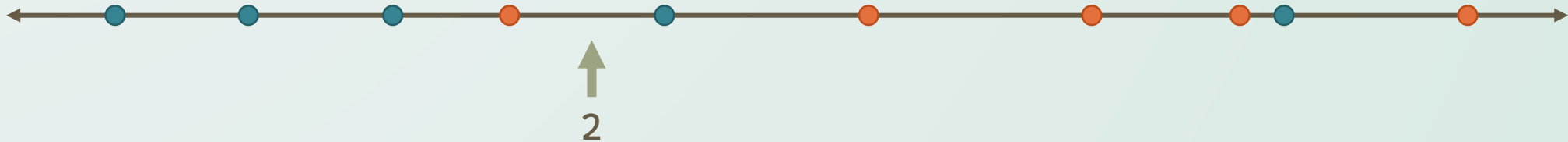
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



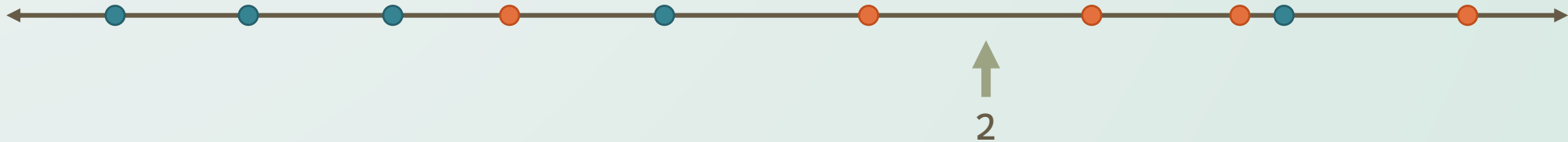
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $O(n \log n)$.
- נעבור על הנקודות בסיבוכיות $O(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



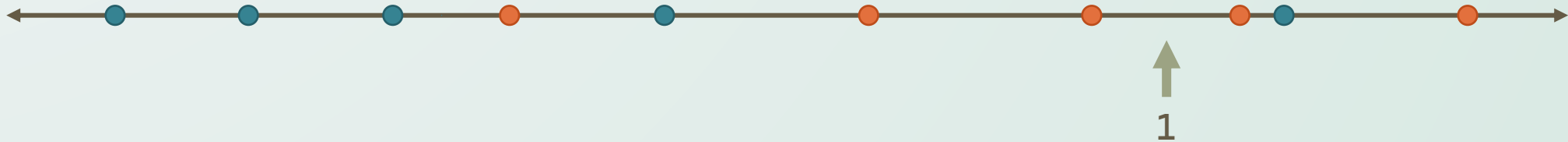
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $O(n \log n)$.
- נעבור על הנקודות בסיבוכיות $O(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



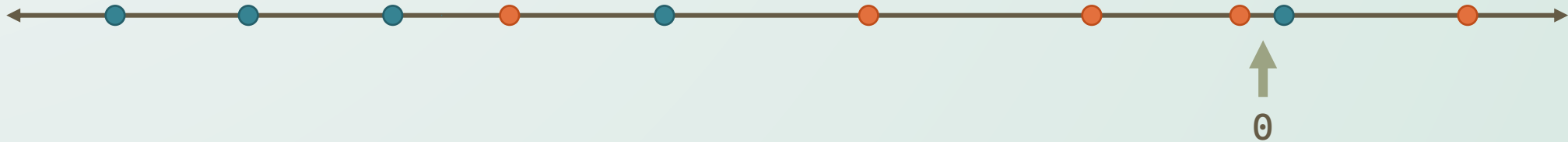
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



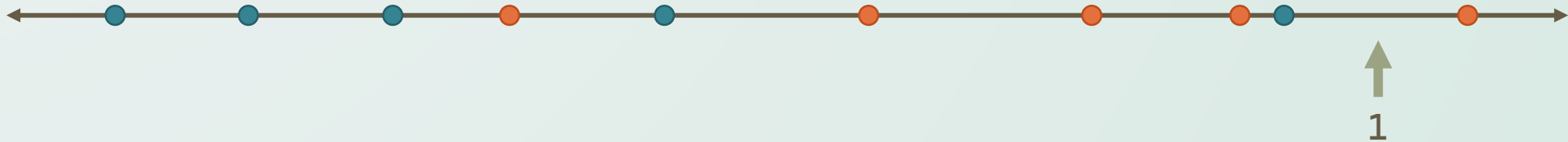
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $O(n \log n)$.
- נעבור על הנקודות בסיבוכיות $O(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



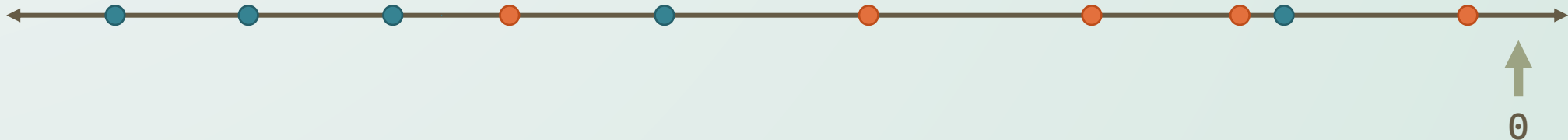
הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



הפתרון המלא

- נרכיב מערך בגודל $2n$ ובו כל הנקודות המעניינות.
- נסמן כל נקודה, האם היא מסמלת כניסה או יציאה של סועד מהמסעדה.
- נמיין את המערך בסיבוכיות $\mathcal{O}(n \log n)$.
- נעבור על הנקודות בסיבוכיות $\mathcal{O}(n)$ ונשמור מונה שבכל רגע נתון מייצג את מספר הסועדים במסעדה.
- מספר הסועדים המקסימלי הינו המקסימום של המונה הנ"ל לאורך כל ריצת התוכנית!



הוכחת נכונות

- הראנו כבר כי מספיק לבדוק רק את $2n$ הנקודות ולקחת את המקסימום בינהן.
- נשאר להוכיח את נכונות פעולת שינוי המונה: נעשה זאת באינדוקציה על מעבר על המערך הממויין.
- **בסיס האינדוקציה:** לפני כל תחילת המערך הממויין, מספר הסועדים במסעדה הינו אפס, מכיוון שאם היה לפחות סועד אחד במסעדה, משמאל לנקודה הנוכחית במערך הייתה צריכה להופיע לפחות נקודת כניסה אחת.
- **צעד האינדוקציה:** נניח כי המונה נכון לפני הצעד ה- i , ונוכיח כי הוא נכון גם אחריו. ובכן, אם הנקודה היא נקודת כניסה, אחרי הנקודה יהיו במסעדה בדיוק $1+$ סועדים. אחרת, זוהי נקודת יציאה ויהיו במסעדה בדיוק $1-$ סועדים. לכן המונה נשאר תקין אחרי הצעד ה- i , כנדרש.

בעיית החציון הממודג

בעיית החציון הממוזג

- לפניכם שני מערכים **ממוינים** $a_1, a_2, \dots, a_n$ ו- $b_1, b_2, \dots, b_n$ באורך n כל אחד. תארו אלגוריתם למציאת החציון של המיזוג בין שני המערכים.

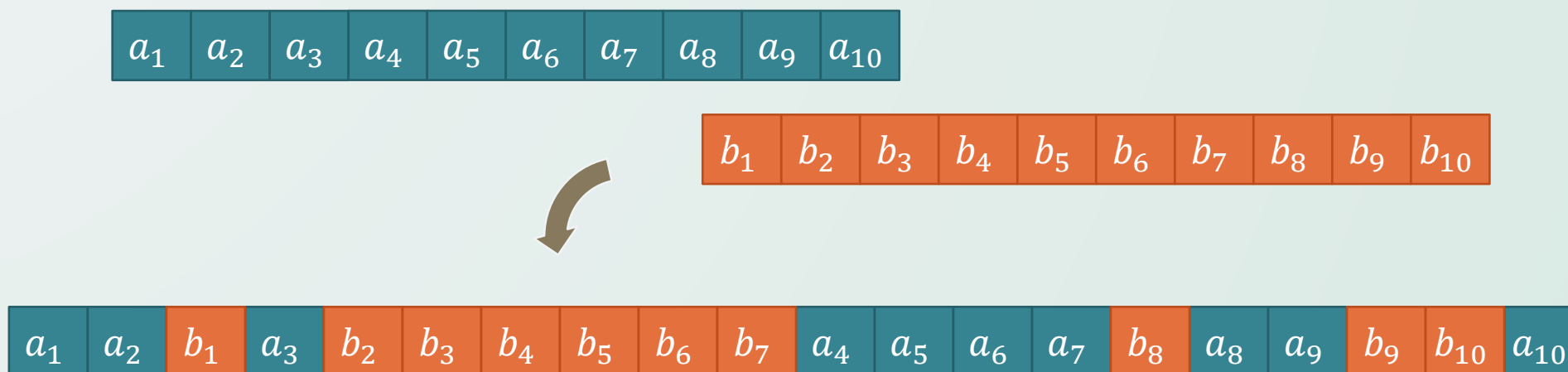
a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

- נגדיר את האיבר החציוני של קבוצה בגודל $2n$ כאיבר ה- n בסידור.
- ניתן להניח כי כל $2n$ האיברים שונים אחד מהשני.

בעיית החציון הממוזג

- לפניכם שני מערכים **ממוינים** $a_1, a_2, \dots, a_n$ ו- $b_1, b_2, \dots, b_n$ באורך n כל אחד. תארו אלגוריתם למציאת החציון של המיזוג בין שני המערכים.



- נגדיר את האיבר החציוני של קבוצה בגודל $2n$ כאיבר ה- n בסידור.
- ניתן להניח כי כל $2n$ האיברים שונים אחד מהשני.

בעיית החציון הממוזג

- לפניכם שני מערכים **ממוינים** $a_1, a_2, \dots, a_n$ ו- $b_1, b_2, \dots, b_n$ באורך n כל אחד. תארו אלגוריתם למציאת החציון של המיזוג בין שני המערכים.

a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

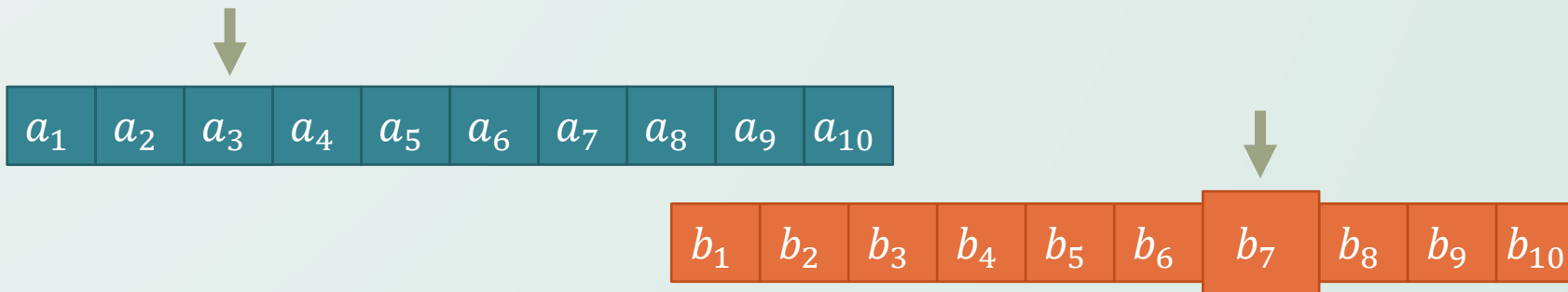


a_1	a_2	b_1	a_3	b_2	b_3	b_4	b_5	b_6	b_7	a_4	a_5	a_6	a_7	b_8	a_8	a_9	b_9	b_{10}	a_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	----------	----------

- נגדיר את האיבר החציוני של קבוצה בגודל $2n$ כאיבר ה- $n+1$ בסידור.
- ניתן להניח כי כל $2n$ האיברים שונים אחד מהשני.

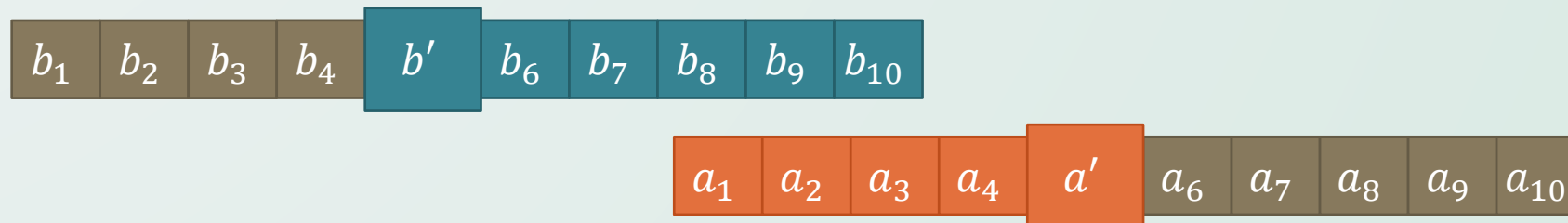
הפתרון הטריטוריאלי

- ניתן לעבור על שני המערכים במקביל, ולהרכיב את המערך הממויין החדש.
- לכן סיבוכיות הפתרון הטריטוריאלי הינה $O(n)$.
- האם ישנו פתרון שסיבוכיותו $O(n)$? כלומר תת-לינארית?



הבחנות בדרך לפתרון

- נשווה בין שני החציונים $a' = a_{n/2}$ ו- $b' = b_{n/2}$ ונניח בה"כ כי $a' < b'$.
- מה נוכל להגיד על האיברים $a_1, \dots, a'$ ועל $b', \dots, b_n$?



הפתרון האופטימלי

- בעזרת השוואת שני החציונים בטווח מסוים של המערכים, נוכל לפסול חצי מהאיברים כמועמדים לחציון הכולל.
- לאחר מכן, נוכל להריץ את האלגוריתם באופן רקורסיבי על מערכים בגודל $n/2$ כל אחד.
- סיבוכיות הפתרון הינה:

$$T(n) = \mathcal{O}(1) + T(n/2) = \mathcal{O}(\log n)$$

כלומר $T(n) \in o(n)$ כנדרש!

הקדמה קלה לגרפים

- גרף $G = (V, E)$ הוא קבוצה של צמתים המחוברות אחד לשני באמצעות קשתות. גרף יקרא **מכוון** אם קשת $u \rightarrow v$ משמע אפשר לעבור מ u ל v אך לא להיפך.
- בגרף לא מכוון, יש סימטריה.

קצת משחקים

- נתבונן במשחק הבא. נתון גרף לא מכוון, ושני שחקנים – רמי ודינה. כל אחד מהם נמצא על קודקוד בגרף, והם יכולים לנוע רק דרך הקשתות.

קצת משחקים

- נתבונן במשחק הבא. נתון גרף לא מכוון, ושני שחקנים – רמי ודינה. כל אחד מהם נמצא על קודקוד בגרף, והם יכולים לנוע רק דרך הקשתות.
- בתורו של שחקן מסוים, הוא יכול לנוע על קשת לצומת סמוכה, או להישאר במקום.
- המטרה של רמי, היא להגיע לאותו הקודקוד כמו של רינה. אם הוא הצליח הוא ניצח.

קצת משחקים

- נתבונן במשחק הבא. נתון גרף לא מכוון, ושני שחקנים – רמי ודינה. כל אחד מהם נמצא על קודקוד בגרף, והם יכולים לנוע רק דרך הקשתות.
- בתורו של שחקן מסוים, הוא יכול לנוע על קשת לצומת סמוכה, או להישאר במקום.
- המטרה של רמי, היא להגיע לאותו הקודקוד כמו של רינה. אם הוא הצליח הוא ניצח.
- המטרה של דינה, למנוע בכל מחיר את הגעתו של רמי לאותו הקודקוד.

קצת משחקים

- נתבונן במשחק הבא. נתון גרף לא מכוון, ושני שחקנים – רמי ודינה. כל אחד מהם נמצא על קודקוד בגרף, והם יכולים לנוע רק דרך הקשתות.
- בתורו של שחקן מסוים, הוא יכול לנוע על קשת לצומת סמוכה, או להישאר במקום.
- המטרה של רמי, היא להגיע לאותו הקודקוד כמו של רינה. אם הוא הצליח הוא ניצח.
- המטרה של דינה, למנוע בכל מחיר את הגעתו של רמי לאותו הקודקוד.
- תארו אלגוריתם פולינומי, אשר בהינתן הגרף, המיקום ההתחלתי של רמי ודינה, ומי מתחיל:
 - מחזיר האם לרמי יש אסטרטגיה מנצחת
 - מאפשר מענה ב $O(1)$ לצעד הבא הטוב ביותר. לדוגמא, אם רמי מתחיל, יחזיר את המהלך האופטימלי, ובהינתן המהלך הבא של דינה, יחזיר שוב את המהלך האופטימלי וכן הלאה. (כמו לצפות בשני מחשבים משחקים שחמט)
- הדרכה: נסו ליצור גרף המתאר את המשחק. מה תהיה תכונה מיוחדת של הגרף? איך מבדילים בין תורות? מה כל צומת מייצג? מה כל קשת מייצגת?

אבחנה ראשונה

• **שאלה** – האם שווה לרמי לא לזוז?

אבחנה ראשונה

- **שאלה** – האם שווה לרמי לא לזוז?
- **תשובה** – לא. אם הוא לא זז, דינה יכולה גם לא לזוז וחזרנו לאותו המצב. לכן רמי תמיד זז.

אבחנה ראשונה

- **שאלה** – האם שווה לרמי לא לזוז?
- **תשובה** – לא. אם הוא לא זז, דינה יכולה גם לא לזוז וחזרנו לאותו המצב. לכן רמי תמיד זז.
- לצערנו, האבחנה הזאת לא מקדמת אותנו במיוחד, אבל כן הייתה יכולה כנראה לעזור אם היינו הולכים בגישה אחרת (שאלה למחשבה – האם יש אלגוריתם שבו האבחנה הזאת תעזור?)

הפתרון

- נבנה גרף דו-צדדי המייצג את המשחק, צד אחד משמע תורו של רמי, והצד השני משמע תורה של דינה.

הפתרון

- נבנה גרף דו-צדדי המייצג את המשחק, צד אחד משמע תורו של רמי, והצד השני משמע תורה של דינה.
- בכל צד, נשמור צומת לכל סידור אפשרי של רמי ודינה. כמה צמתים יש בכל צד?

הפתרון

- נבנה גרף דו-צדדי המייצג את המשחק, צד אחד משמע תורו של רמי, והצד השני משמע תורה של דינה.
- בכל צד, נשמור צומת לכל סידור אפשרי של רמי ודינה. כמה צמתים יש בכל צד?
- תשובה - n^2 צמתים – כל אחד מהם יכול להיות בכל אחת מ n צמתים.

הפתרון

- נבנה גרף דו-צדדי המייצג את המשחק, צד אחד משמע תורו של רמי, והצד השני משמע תורה של דינה.
- בכל צד, נשמור צומת לכל סידור אפשרי של רמי ודינה. כמה צמתים יש בכל צד?
- תשובה - n^2 צמתים – כל אחד מהם יכול להיות בכל אחת מ- n צמתים.
- קשת בין שני צמתים תהיה בגרף אמ"מ אפשר לעבור בין שני המצבים על ידי צעד כלשהו.
- אם היינו בצד של רמי והיה תור, עכשיו יהיה תורה של דינה, ולהיפך.
- כלומר הגרף הוא באמת דו-צדדי.

הפתרון - המשך

- **תזכורת:** גרף דו"צ, כל צד מייצג תור של שחקן אחר, כל צומת מייצגת מצב, וכל קשת מסמנת שאפשר לעבור בין המצבים.
- נסמן צומת **שחורה** אם ממנה בטוח רמי יכול להכריח ניצחון.
- ברור שכל צומת המייצגת מצב שרמי ודינה על אותה הצומת, תהיה צומת שחורה.

הפתרון - המשך

- **תזכורת:** גרף דו"צ, כל צד מייצג תור של שחקן אחר, כל צומת מייצגת מצב, וכל קשת מסמנת שאפשר לעבור בין המצבים.
- נסמן צומת **שחורה** אם ממנה בטוח רמי יכול להכריח ניצחון.
- ברור שכל צומת המייצגת מצב שרמי ודינה על אותה הצומת, תהיה צומת שחורה.
- אם אנחנו בצד של רמי, ויש קשת לצומת שחורה – הרי שרמי יכול לעשות את הצעד המתאים, ומשם מובטח לנו שרמי יכול להכריח ניצחון. כלומר, כל צומת בצד של רמי עם צומת לקשת שחורה – היא שחורה בעצמה.

הפתרון - המשך

- **תזכורת:** גרף דו"צ, כל צד מייצג תור של שחקן אחר, כל צומת מייצגת מצב, וכל קשת מסמנת שאפשר לעבור בין המצבים.
- נסמן צומת **שחורה** אם ממנה בטוח רמי יכול להכריח ניצחון.
- ברור שכל צומת המייצגת מצב שרמי ודינה על אותה הצומת, תהיה צומת שחורה.
- אם אנחנו בצד של רמי, ויש קשת לצומת שחורה – הרי שרמי יכול לעשות את הצעד המתאים, ומשם מובטח לנו שרמי יכול להכריח ניצחון. כלומר, כל צומת בצד של רמי עם צומת לקשת שחורה – היא שחורה בעצמה.
- אם אנחנו בצד של דינה, וכל הקשתות הן לצומת שחורה – הרי דינה חייבת לעשות צעד שממנו רמי יוכל להכריח ניצחון. לכן זו גם תהיה צומת שחורה.

הפתרון - המשך

- **תזכורת:** גרף דו"צ, כל צד מייצג תור של שחקן אחר, כל צומת מייצגת מצב, וכל קשת מסמנת שאפשר לעבור בין המצבים.
- נסמן צומת **שחורה** אם ממנה בטוח רמי יכול להכריח ניצחון.
- ברור שכל צומת המייצגת מצב שרמי ודינה על אותה הצומת, תהיה צומת שחורה.
- אם אנחנו בצד של רמי, ויש קשת לצומת שחורה – הרי שרמי יכול לעשות את הצעד המתאים, ומשם מובטח לנו שרמי יכול להכריח ניצחון. כלומר, כל צומת בצד של רמי עם צומת לקשת שחורה – היא שחורה בעצמה.
- אם אנחנו בצד של דינה, וכל הקשתות הן לצומת שחורה – הרי דינה חייבת לעשות צעד שממנו רמי יוכל להכריח ניצחון. לכן זו גם תהיה צומת שחורה.
- עד פה האבחנות החשובות, שמובילות מיד לאלגוריתם. מישהו רוצה להציע?

הפתרון - אלגוריתם

- ראשית בונים את הגרף הדו"צ.
- עוברים על כל הצמתים המייצגות מצב מנצח, כלומר רמי ודינה על אותה הצומת, ומשחירים.

הפתרון - אלגוריתם

- ראשית בונים את הגרף הדו"צ.
- עוברים על כל הצמתים המייצגות מצב מנצח, כלומר רמי ודינה על אותה הצומת, ומשחירים.
- לכל צומת בצד של רמי, בודקים האם יש לה קשת לצומת שחורה. אם כן, משחירים.
- לכל צומת בצד של דינה, בודקים האם כל השכנים שלה שחורים. אם כן, משחירים.

הפתרון - אלגוריתם

- ראשית בונים את הגרף הדו"צ.
- עוברים על כל הצמתים המייצגות מצב מנצח, כלומר רמי ודינה על אותה הצומת, ומשחירים.
- לכל צומת בצד של רמי, בודקים האם יש לה קשת לצומת שחורה. אם כן, משחירים.
- לכל צומת בצד של דינה, בודקים האם כל השכנים שלה שחורים. אם כן, משחירים.
- חוזרים על שני השלבים האחרונים (עד מתי?)

הפתרון - אלגוריתם

- ראשית בונים את הגרף הדו"צ.
- עוברים על כל הצמתים המייצגות מצב מנצח, כלומר רמי ודינה על אותה הצומת, ומשחירים.
- לכל צומת בצד של רמי, בודקים האם יש לה קשת לצומת שחורה. אם כן, משחירים.
- לכל צומת בצד של דינה, בודקים האם כל השכנים שלה שחורים. אם כן, משחירים.
- חוזרים על שני השלבים האחרונים (עד מתי?) עד שלא השחרנו אף צומת חדשה.
- כעת סיימנו את ההכנה. אם רוצים רק לדעת האם רמי יכול להכריח ניצחון, ניגשים לצומת המתאימה, ובודקים האם היא שחורה.
- על מנת להחזיר מה הצעד הכי טוב, נצטרך לשנות משהו קטן. תרגיל – למה עכשיו זה לא עובד? מה נשנה? (תשובה בהערות)

הפתרון - סיבוכיות

- בכל צד יש n^2 צמתים. בכל איטרציה, עוברים על כולם. כיוון שמובטח שבכל איטרציה (למעט האחרונה) משחירים מישו, יש לכל היותר $2n^2$ איטרציות. כלומר $O(n^4)$ עבודה סה"כ.

הפתרון - סיבוכיות

- בכל צד יש n^2 צמתים. בכל איטרציה, עוברים על כולם. כיוון שמובטח שבכל איטרציה (למעט האחרונה) משחירים מישו, יש לכל היותר $2n^2$ איטרציות. כלומר $O(n^4)$ עבודה סה"כ.
- לגבי הוכחת הנכונות – תנסו להוכיח פורמלית בבית.